Fresco: A Permissioned Blockchain Framework for Securing Model Context Protocol

Zhankai Ye[1], Yusen Wu[2], Shixian Sheng[2], Phuong Nguyen[2],
Yili Ren[3], Bingyang Wei[4], Yelena Yesha[2], Xin Liu[1,‡]
[1]Florida State University, [2]University of Miami, [3]University of South Florida, [4]Texas Christian University
{zy22b, xliu15}@fsu.edu, {yxw1259, pxn208, yxy806}@miami.edu,
shixian_sheng-2@protonmail.com, yiliren@usf.edu, b.wei@tcu.edu

***Abstract*—The increasing integration of Large Language Models (LLMs) with external tools and data sources via the Model Context Protocol (MCP) introduces significant security and privacy challenges, including identity mismanagement, data exfiltration, and compositional risks. Current MCP setups often lack robust, unified security mechanisms, leading to vulnerabilities. This paper proposes Fresco, a novel framework that leverages a permissioned blockchain to establish a secure, transparent, and auditable layer for MCP-based LLM interactions. Fresco redefines the MCP server using on-chain smart contracts for verifiable logic, incorporates a comprehensive access control system with role-bound certificates and capability tokens, and employs a multi-stage policy checker with decentralized invocation sanitization (including fine-tuned LLM-based anonymization) and hidden executor selection. The framework is underpinned by a BFT consensus mechanism running in trusted execution environments (AWS Nitro Enclaves) for integrity, a Parallel DAG Ledger (PDL) for high-throughput auditable logging, and a threshold-encrypted storage system for privacy-preserving data handling. Our evaluation shows that Fresco effectively mitigates key security risks. Furthermore, Fresco achieves sub-50 ms latency for local read access, up to 1,500 TPS in LAN environments, and 1,500–3,000 TPM with LLM integration, while its audit database supports 40,000 operations per second with 30–35 ms latency.**

I. Introduction

LLMs remain limited by static pretraining unless they can access fresh enterprise data and external tools. The MCP addresses this integration gap by standardizing how a client discovers tools, invokes them through MCP servers, and returns results to the model, replacing bespoke connectors with a modular interface [7]. In practice, MCP lets heterogeneous backends appear as reusable services, improving maintainability and deployment flexibility [7], [17], [9].

This same modularity expands the attack surface: requests cross organizational boundaries, tool metadata influences model behavior, and authorization decisions must hold across chained invocations [15], [10]. We focus on three risk classes and the concrete control each requires:

- **Identity and Permission Mismanagement.** MCP deployments often authenticate endpoints loosely and treat tool metadata as trustworthy, enabling *tool impersonation attacks* [35], [19]. They also risk violating least privilege when long-lived or coarse-grained permissions are reused across requests, creating *overprivilege vulnerabilities* [33], [31]. Mitigation therefore requires authenticated principals, explicit tool registration, and narrowly scoped capabilities.
- **Data Exfiltration via Embedded Malicious Logic.** Attackers can place malicious instructions in prompts or tool metadata so that the model performs covert actions. Typical examples are *prompt injection* and *poisoned tools*, which can trigger unauthorized reads, secret extraction, or silent data upload [24], [19], [36]. Mitigation requires both policy checks on invocation content and auditable execution records.
- **Compositional Security Risks.** Even when individual clients, servers, tools, and models look secure in isolation, chaining them creates new failure modes such as *cross-model prompt injection* and *prompt infection* in multi-agent workflows [39], [25]. Mitigation requires provenance, replayable logs, and enforcement points that span the full invocation path.

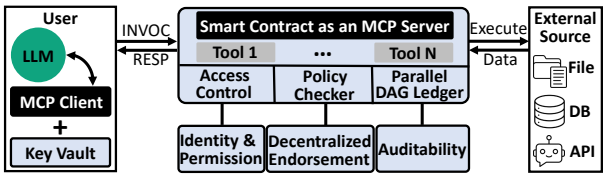

Fig. 1: Fresco integrates blockchain-based components, including a smart contract MCP server, role-based access control, a decentralized policy checker, and a PDL to ensure secure, resilient, and transparent interactions in the MCP framework.

To address these risks, we propose Fresco, a permissioned blockchain framework for MCP deployments that span partially trusted administrative domains [20], [11]. Our goal is not to replace every centralized deployment, but to provide a setting where multiple parties can share integrity, authorization state, and audit evidence without relying on one operator to maintain the reference log.

The core design redefines the MCP server as contract-governed logic with explicit registration and authorization. As shown in Fig. 1, Fresco uses role-bound certificates, tool registration, lease-style capability tokens, and controlled delegation to enforce least privilege at invocation time. It then applies a multi-stage Policy Checker that combines static checks, LLM-guided semantic filtering, hidden executor selection, and auditable logging to reduce malicious or policy-violating

‡ Corresponding Author

executions.

These controls are backed by a permissioned blockchain designed for integrity and auditability. A customized BFT protocol fortified by AWS Nitro Enclaves secures state transitions and policy enforcement [14]; a PDL records compact policy and access metadata for tamper-evident audit; and threshold-encrypted storage protects sensitive responses off-chain.

This paper makes the following contributions:

- We propose Fresco, a permissioned blockchain-based framework for securing MCP-mediated LLM interactions across partially trusted organizations.
- Fresco couples least-privilege access control, policy-based invocation filtering, and verifiable audit logging in one end-to-end design.
- We show that Fresco reduces policy bypass under our evaluation setup while sustaining sub-50 ms local reads, up to 1,500 TPS in LAN, 1,500–3,000 TPM with LLM integration, and about 40,000 audit-database operations per second.

II. Background

A. Model Context Protocol

The MCP is an open standard for connecting AI applications to external tools and data sources through a uniform interface [7]. A typical MCP deployment includes a **User**, an **MCP Client**, and an **MCP Server**: the User issues a request, the client manages protocol communication, and the server exposes tool capabilities backed by external systems. In a standard workflow, the client first establishes a session and discovers available capabilities, then sends a tool invocation to the server, which executes the requested operation against an external source and returns the result for the LLM to integrate into its final response.

B. Permissioned Blockchain

Permissioned blockchains are decentralized ledger systems in which access to participation is restricted to a defined group of entities. A typical permissioned blockchain network includes several organizations or departments, each hosting one or more nodes. These nodes participate in consensus, validate transactions, and maintain synchronized copies of the ledger. Smart contracts deployed on the chain define the permissible operations, enforce access policies, and enable automation across the network [5].

Unlike public blockchains, permissioned blockchains operate in a controlled setting with authenticated participants and lower consensus overhead [6], [18]. This makes them well suited for enterprise MCP deployments, where the system must enforce participant-level access control, support collaboration across multiple organizations, and provide low-latency BFT-style ordering [43]. Because members are identifiable and all state transitions are tamper-evident, permissioned blockchains offer a practical foundation for secure and auditable coordination among semitrusted parties.

III. Threat Model and Assumption

Fresco targets cross-organization enterprise deployments in which each participant maintains one or more nodes in a permissioned blockchain network. These nodes support MCP-mediated access to external data and may initiate, endorse, or execute requests on behalf of local clients.

Adversarial capabilities. A malicious node or client may attempt the following:

- **Identity spoofing or permission misuse**, such as impersonating tools or clients to gain unauthorized access.
- **Sensitive input injection**, embedding covert instructions or identifiers (e.g., prompt injection, poisoned tool descriptors) to exfiltrate data.
- **Audit evasion**, by denying involvement in a request or attempting to tamper with historical logs or access traces.
- **Linkability inference**, correlating request patterns with specific users or organizations to compromise privacy.

Security goals. Fresco is designed to:

- Authenticate and authorize all MCP requests via on-chain access control with least-privilege capability scopes;
- Sanitize and validate all invocations against malicious content before execution;
- Ensure tamper-evident logging of all interactions for forensic accountability;
- Hide executor identities and isolate execution to prevent targeted attacks.

Trust assumptions. We assume that all nodes belong to identifiable, pre-approved entities. Nodes are semi-trusted: they may infer sensitive information, evade audit, or abuse legitimate permissions, but they do not break the authenticated channel or compromise the enclave runtime by assumption. We therefore do not model active network-level adversaries or open Internet enrollment. The BFT layer, executed in trusted enclaves (e.g., AWS Nitro), is assumed to provide integrity and non-equivocation. Accordingly, our guarantees are intended for controlled enterprise settings rather than arbitrary public MCP deployments.

IV. Design of Fresco

A. Smart Contract as an MCP Server

A core architectural feature of Fresco is that the MCP server is realized as on-chain smart contracts rather than as a standalone centralized service. This shifts server-side control into a verifiable execution environment backed by immutable contract logic and decentralized agreement, giving MCP invocations stronger transparency, resilience, and auditability.

The MCP server is implemented as a Go-based smart contract deployed on a permissioned blockchain. As shown in Fig. 2, a client submits each tool invocation as a structured request containing identity metadata and invocation content.

The contract first enforces access control by authenticating the client, validating the requested tool, and checking its authorization scope (Section IV-B). It then forwards the content to the policy checker, where peers perform decentralized sanitization and return signed endorsements (Section IV-C).

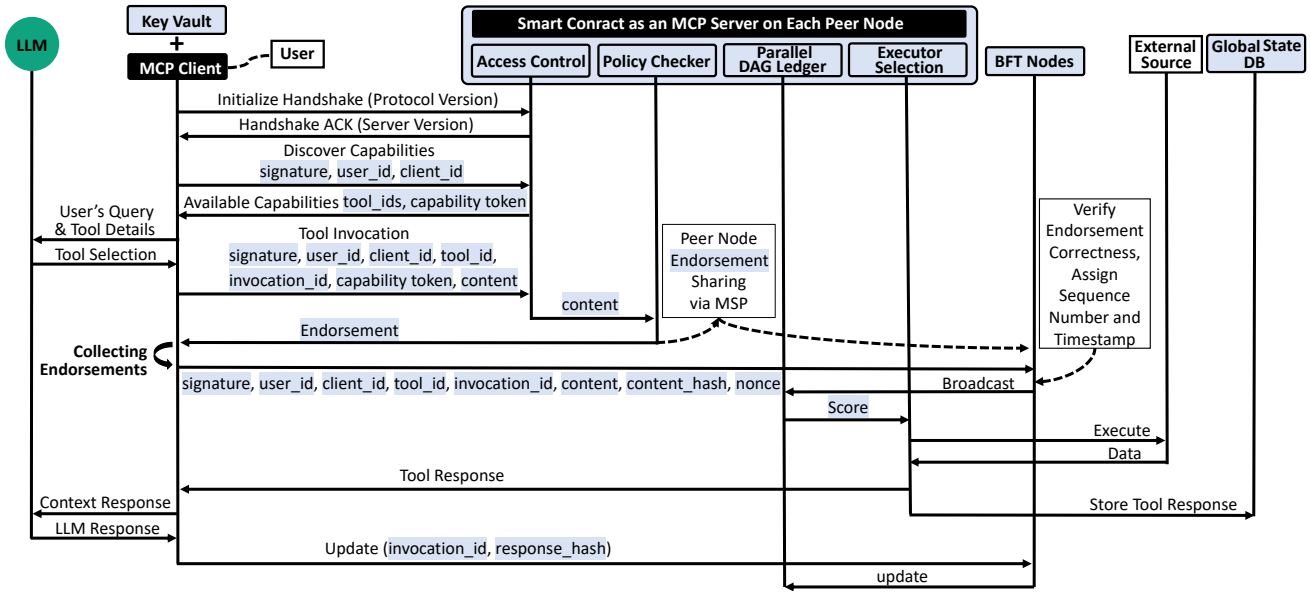

Fig. 2: End-to-end MCP invocation workflow in Fresco. The diagram illustrates how an LLM interacts with external tools through the MCP client and smart contract–based MCP server. The process includes access control, policy checking, endorsement collection, executor selection via BFT consensus, and logging of all metadata to the PDL and global state database.

After the client collects a quorum of endorsements, it submits them on-chain for final verification. If all identity, permission, and policy checks succeed, the system deterministically selects a hidden executor to invoke the external API (Section IV-C1), while response data are stored in the threshold-protected global state database (Section IV-E).

Throughout this pipeline, metadata for both successful and failed checks are committed to the PDL, providing an auditable trace of identity verification, policy enforcement, and execution outcomes. The ledger is optimized for concurrent writes and efficient audit queries (Section IV-D).

Although Fresco implements the MCP server with smart contracts, it preserves the original MCP interface and invocation semantics. The main change is the execution substrate: server-side decisions are enforced by decentralized, verifiable infrastructure rather than by a single trusted service.

B. Access Control

Fresco's access-control subsystem combines identity verification, tool validation, short-lived capability tokens, and controlled delegation in one on-chain mechanism. Its purpose is to determine which authenticated client may invoke which registered tool under what scope.

Identity assurance through role-bound certificates. Each client registers with an on-chain Certificate Authority (CA) and obtains an X.509 certificate binding its organizational identity to one of four roles: `submitter`, `auditor`, `proxy`, or `verifier`. These roles respectively govern request submission, log inspection, delegated operation, and independent validation. For each invocation, peer nodes use the Membership Service Provider (MSP) to verify certificate authenticity and expiration before any tool-level authorization is applied.

Tool verification and authorisation. To prevent tool impersonation and supply-chain abuse, every invocable tool must be registered in a `ToolRegistry` contract and approved before activation. During `InvokeTool`, the contract checks both the tool's approval status and whether the caller's role is authorized for that tool.

Lease-style capability tokens. Once identity and tool validation succeed, the contract verifies a signed short-term *Capability Token* that binds the caller to a specific tool under bounded lifetime, usage count, and delegation permissions. Expired or exhausted tokens are rejected, preventing stale privileges from being silently reused.

Controlled delegation. When delegation is permitted, the original token must explicitly allow it. The delegate presents its own certificate together with the parent token, and the contract records the relationship on-chain before issuing a child token with stricter limits. Delegation depth can be bounded, and any link in the chain can be revoked immediately, preserving a clear and auditable permission trail.

C. Policy Checker

After validating identity, tool permissions, and capability tokens, the smart contract passes the invocation content to the policy checker. This subsystem enforces content-level compliance in two stages.

Decentralized Invocation Sanitization. The invocation is broadcast to authorized peer nodes, each of which evaluates it inside an isolated containerized policy environment. The sanitization pipeline combines deterministic checks with LLM-guided semantic screening to detect sensitive content and policy violations before endorsement.

The static stage applies a deterministic rule engine that validates the invocation schema and matches sensitive literals using regular expressions and curated dictionaries. Rules are versioned on-chain so that all peers evaluate the same content under the same policy.

The semantic stage invokes a lightweight on-node LLM to detect contextual or obfuscated violations that evade static filtering. The model returns a risk score, and invocations above a configurable threshold are rejected. To preserve policy consistency across peers, model artifacts are fine-tuned offline, signed, and distributed in a controlled form.

1) Executor Selection: To preserve privacy and prevent executor-targeted attacks, Fresco employs a hidden executor selection mechanism that decouples the invocation submission phase from the execution phase.

Each peer independently issues a signed endorsement if the invocation passes policy checks, and the client collects a quorum of valid endorsements.

The client then packages a random `nonce` and the endorsed metadata into a transaction for the BFT layer. Because peer and BFT nodes share a common MSP, the ordering layer can verify the authenticity and quorum of the endorsements before commitment.

At this point, all peer nodes observe the committed transaction and deterministically compute the executor node. Each node independently calculates:

```
score = hash(nonce | client_id | node_id |
            invocation_id)
```

The node with the lowest score becomes the executor and proceeds to execute the external API. Since the `node_id` used in scoring is unknown to the client, the executor's identity remains hidden until execution.

This mechanism yields consistent selection without extra inter-node coordination, while reducing targeted attacks, identity linkage, and traffic-correlation risks.

D. BFT Consensus and Parallel DAG Ledger

Compositional security risks in complex LLM workflows often remain undetected at runtime, making transparent and auditable logging essential. Fresco addresses this by immutably recording metadata from every pipeline step—including identity verification, permission checks, policy compliance, and external execution, regardless of the outcome. These logs enable post-hoc analysis of subtle threats and support reconstruction of system states for forensic auditing. To meet the demands of high-volume, auditable metadata logging, Fresco integrates a BFT consensus layer and a PDL.

1) Efficient Consensus Protocols with AWS Nitro Enclaves: Fresco's coordination relies on a customized version of HotStuff [43]. To strengthen integrity, each BFT replica runs inside an AWS Nitro Enclave, a lightweight TEE that supports attested execution of unmodified Linux applications, including Golang binaries.

BFT consists of $n = 2f + 1$ EC2 instances, each hosting one Nitro Enclave that executes the HotStuff core in isolation. Nitro Enclaves provide hardware-enforced memory and CPU isolation, while communication with the host is mediated through `vsock`.

As shown in Fig. 3, the enclave-assisted protocol follows a HotStuff-style *key-lock-commit* pipeline while reducing the replica count from $3f + 1$ to $2f + 1$ by relying on trusted enclave modules. Unlike a conventional host process, the enclave signs each phase transition with a hardware-protected identity, preventing equivocation by the host OS.

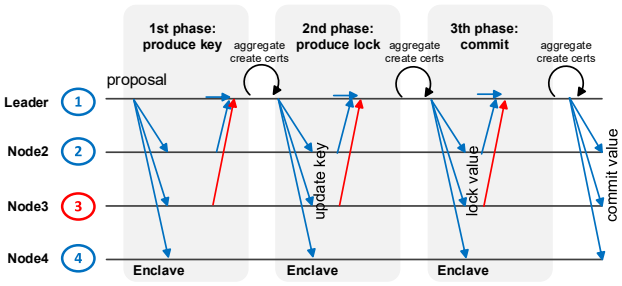

Fig. 3: Enclave-based three-phase BFT consensus.

In each phase, replicas validate the proposal inside the enclave and return enclave-signed votes; once the leader gathers a quorum, it forms the corresponding certificate and advances the protocol. A value is finalized after the commit phase collects the required enclave-signed responses.

Compromised replicas cannot forge valid phase transitions without enclave support, so their messages are rejected by honest nodes. This preserves HotStuff-style pipelining while reducing quorum size and communication overhead.

Replica Requirements. Classical BFT protocols require $n \geq 3f+1$ because Byzantine replicas may equivocate. Here, enclave-enforced vote signing prevents equivocation, allowing the system to operate with:

$$n \geq 2f + 1$$

because any two quorums of size $f + 1$ intersect in at least one honest enclave. This follows the same principle as prior trusted-hardware BFT designs [27], [4].

Fresco ensures key BFT properties:

- **Safety.** Any two quorums intersect in at least one honest, non-equivocating enclave, preventing conflicting commits.
- **Liveness.** Under partial synchrony and a correct leader, honest replicas continue to advance through the three consensus phases.
- **Availability.** Consensus can continue as long as at least $f+1$ enclaves remain operational, even if up to f nodes crash or become unreachable.

2) PDL for High-throughput Transaction Immutability and Auditability: **Ledger structure and edge semantics.** PDL models the ledger as a DAG with multiple sectors (parallel local chains), as shown in Fig. 4. It uses three edge types: *causal edges* $(u \to v)$ for intra-sector order, *audit edges* for cross-sector dependency checks, and *merge edges* for periodic reconciliation of sector tips. This separation enables parallel appends while preserving global consistency.

Concurrency model. Let $G = (V, E_c \cup E_a \cup E_m)$, where transactions are vertices and edges encode causal, audit, and merge dependencies. For transactions x and y, if neither $x \rightsquigarrow y$ nor $y \rightsquigarrow x$, they are concurrent and may be appended

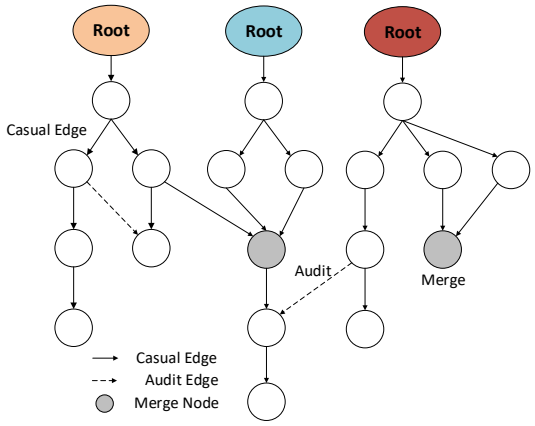

Fig. 4: Illustrates the architecture of the PDL. Each colored "Root" node (Orange, Blue, **Red**) represents a separate sector or organization. These sectors grow independently and append transactions concurrently, forming their own local chains. Solid arrows indicate causal edges, which represent strict execution order within a sector. These edges enforce that a transaction must follow its parent and capture strong dependencies. Dashed arrows represent audit edges, which link transactions across sectors. These edges do not enforce execution order but indicate that one transaction needs to reference or validate another, such as a cross-organization read or context check. The shaded oval node is a merge node, which collects the heads of multiple sectors.

in parallel across sectors; if they conflict on shared state, PDL imposes order through causal or merge edges. If each sector sustains throughput μ and S sectors run concurrently, theoretical throughput scales as $T_{\max} \approx S{\cdot}\mu$ when cross-sector checks are not dominant.

Prompt-aware audit edges. For prompt-driven workflows, audit edges capture semantic dependencies between prompt transactions across sessions and users. We define the *audit cone* of prompt p as all prior transactions that can reach p through causal or audit links:

$$A(p) = \{p' \in V \mid \exists \text{ path } \pi : p' \rightsquigarrow p \text{ in } G\} \tag{1}$$

Each audit edge stores a certified reference, so verification checks signatures along $A(p)$ and detects conflicting write paths in $O(|A(p)|)$ time without replaying the entire ledger. This makes cross-prompt dependency claims cryptographically provable and supports provenance-oriented auditing.

Achieving Parallelism without Sacrificing Consistency. PDL's distinguishing feature is its ability to let validators append blocks in parallel on different sectors without waiting for global locks [26], [41]. This is achieved by (1) decoupling data dissemination from final ordering, and (2) integrating local histories through the DAG structure. In PDL, each new block carries with it causal proof of preceding blocks (so there is no equivocation), and also may include cross-sector references for audit purposes. Because each block can reference multiple parents, validators implicitly help finalize past blocks as they issue new ones. In other words, PDL

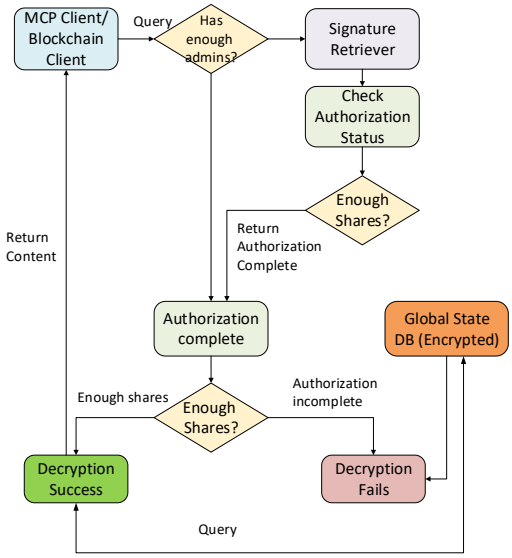

Fig. 5: Global State DB Access Workflow.

performs asynchronous confirmation: each block not only carries new transactions but also votes (by reference) for prior transactions in its ancestry. Crucially, there is no "global lock": a sector can advance its head as long as it respects intra-sector causality, and need only engage other sectors when cross-dependencies arise. Even when merge nodes are inserted, they are created autonomously by validators gathering existing tips, not by a central sequencer. Global consistency is achieved because any conflicting histories will be detected via audit edges and will be resolved by the requirement of quorum-based merging. Effectively, PDL ensures that any two honest views of the ledger differ only in non-final portions; once enough interlinking has occurred, a common prefix of transactions is established across all honest nodes.

E. Global State Database

We integrate a global state database as a privacy-preserving context storage module for Fresco. It supports decentralized confidentiality and client access control using threshold encryption, ensuring that *no single client* can access stored data unilaterally.

1) Threshold Encryption for Secure Access: Fresco employs a (t, n) *threshold encryption* scheme extended with *vector-input labels* (VIL), enabling attribute-based access control per message. Each sensitive message is encrypted under a label $L = (\mathsf{cid}, \mathsf{ts}, \mathsf{op}, \mathsf{ac})$ that binds ciphertext access to client identity, time, operation type, and access-control policy.

2) Algorithms.: The scheme generates public and verification keys together with distributed private-key shares, encrypts data under the label vector, and releases plaintext only when at least t valid decryption shares are combined. No individual administrator can decrypt a stored response alone.

3) Query Access Control: Decryption shares are issued only when the requesting client satisfies the access predicate encoded in ac. This enforces authorization while maintaining

semantic security against adversaries that corrupt fewer than t administrators.

4) Workflow of Context Storage: As shown in Fig. 5, a client submits a `ReadContext` request to the smart contract, which checks whether the request already carries enough valid administrative approvals. If not, the contract invokes a signature-retrieval step that broadcasts the pending request to eligible administrators. Once the threshold is satisfied, the client can retrieve and decrypt the stored ciphertext. All authorization attempts and related metadata are recorded in the PDL to preserve an immutable audit trail.

V. Security and Complexity Analysis

A. Security Analysis

We summarize the key security properties satisfied by Fresco under the threat model in Section III.

BFT properties. With enclave-backed replicas and $n = 2f+1$ nodes, Fresco preserves (i) *safety*: honest replicas never decide conflicting values; (ii) *liveness*: every valid request is eventually committed under eventual synchrony; and (iii) *availability*: progress continues as long as at least $f + 1$ uncompromised replicas remain online.

Under these assumptions, Fresco provides three system-level guarantees. *Authentication* means every accepted invocation carries a valid certificate chain, role binding, and unexpired capability token. *Policy compliance* means an invocation is executed only if its sanitized input satisfies policy π and receives quorum endorsement. *Auditability* means every security-relevant event, including failed checks, is committed to an append-only BFT ledger and remains verifiable after the fact. Together, these properties prevent unauthorized invocations, narrow over-privileged tool use, block policy-violating requests, and make committed traces tamper-evident.

B. Complexity Analysis

For a network with n peer nodes and an ordering committee of size $m = 2f+1$, a tool invocation incurs $O(n+m)$ communication: peers participate in endorsement and the BFT layer finalizes ordering and logging. If a request spans k channels, coordination grows linearly to $O(kn)$. Let L denote the input size. Signature and token checks are constant-time per request, while static sanitization is $O(L)$ per validator. The semantic checker adds model-dependent inference cost; operationally, end-to-end latency is therefore dominated by LLM inference when Layer 2 is invoked, consistent with Section VII. Nitro enclave setup and attestation add deployment and runtime overheads that are not asymptotically dominant but appear as practical constant factors in write latency.

VI. System Implementation

Fresco is implemented as a geo-distributed system tested across multiple Amazon EC2 instances and local network, with compute-intensive components deployed on A100 GPUs. The system preserves the standard MCP client-facing interface, so existing clients interact with Fresco through the same discovery and invocation flow while the server-side logic is replaced by the contract-governed backend. The system comprises four tightly integrated components. First, the consensus layer is built on a HotStuff-based Byzantine Fault Tolerant protocol written in Go, with each replica operating inside an AWS Nitro Enclave [3]. Each enclave holds a sealed private key for signing consensus votes and performs remote attestation verification via vsock before accepting messages, ensuring hardware-backed trust and message integrity. We enclave only the consensus and signing path, not the full storage or LLM stack, which limits both Nitro-specific memory constraints and deployment cost. Second, the smart contract engine adopts a modular design using Go plugins, with each plugin implementing deterministic handlers responsible for access control, logging, and orchestrating interactions through the MCP. These handlers ensure reproducibility and verifiability across all replicas. Third, we apply two-layer sanitization for endorsement to protect against inference attacks: structured sensitive fields are first redacted using Microsoft Presidio (Layer 1) for fast rejection of endorsements as it is static, followed by contextual anonymization via one of a fine-tuned LLM (Layer 2) deployed across the A100 GPU nodes. Each LLM is optimized for a specific anonymization scenario (e.g., healthcare, legal, personal messages, and free-text) to maximize coverage and robustness. Finally, Fresco includes a privacy-preserving global state storage called **FrescoDB**. A global state management module maintains cross-contract execution context using a distributed key-value store, allowing for persistent stateful workflows. The MCP response and content are then protected using threshold encryption [8], requiring t-of-n administrator keys for decryption. By leveraging these designs, Fresco supports high-throughput, low-latency, and auditable MCP request handling at scale via a trusted blockchain system.

VII. Experimental Results

We evaluate Fresco around four questions: **(RQ1)** how effectively the policy checker sanitizes MCP requests before endorsement; **(RQ2)** the end-to-end latency and throughput of the blockchain layer under LAN and WAN conditions; **(RQ3)** the efficiency of FrescoDB for auditable storage operations; and **(RQ4)** whether Fresco provides stronger security guarantees than common centralized LLM orchestration frameworks with acceptable performance overhead.

A. Access Control Effectiveness

To answer **RQ1**, we evaluate access-control enforcement and the associated verification overhead under a workload of 209,261 MCP transactions across 48 registered tools. The on-chain CA and ToolRegistry contracts authenticated 100% of legitimate certificates and rejected 98.7% of unauthorized invocations (e.g., stale or mismatched capability tokens); median verification latency was 3.8 ms in LAN and 17.5 ms in WAN. Lease-style capability tokens further constrained tool usage: 91.3% of denials were due to expired or overused tokens.

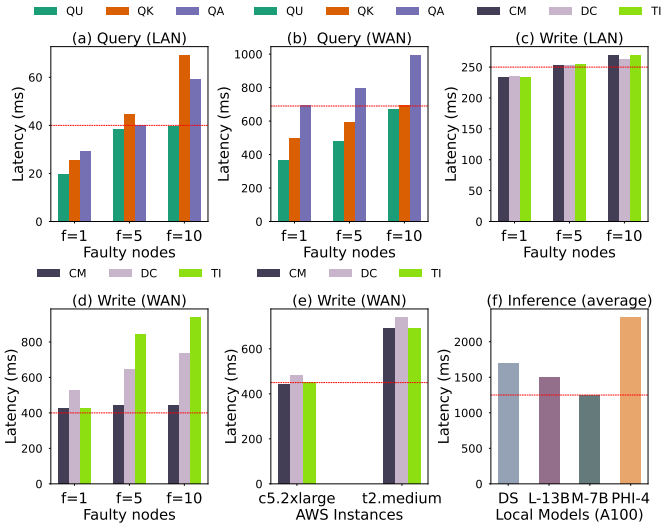

Fig. 6: Fresco Latency.

Abbrev: **QU**=QueryUserDataByID; **QK**=QueryUserDataByKeyword; **QA**=QueryAllUserData; **CM**=CreateMCP; **DC**=DiscoverCapability; **TI**=ToolInvocation.

TABLE I: 5 sample categories accurately identified by the policy checker in the semantic sanitization stage. The results are tested on **209,261** open-sourced transactions.

Category	Policy Checker Precision	F1 Score	Accuracy
EMAIL	0.99	0.96	0.95
MAC	0.97	0.94	0.92
USERAGENT	0.94	0.91	0.90
GPS Location	0.93	0.88	0.87
URL	0.91	0.86	0.84

B. Policy Checker Effectiveness

To assess the effectiveness of Fresco 's layered policy checker, we analyze its behavior across staged sanitization. In the first stage, static sanitization filters over **67%** of requests within milliseconds based on pattern matches against structured literals such as hardcoded credentials, IP addresses, and numeric identifiers. This fast-path rejection reduces downstream workload and near-eliminates cost for clearly invalid requests.

The second stage employs a fine-tuned on-node LLM (evaluated with four models) to identify context-rich policy violations that escape static checks. In sensitive fields where precision is critical, the LLM achieves **0.96** F1 on EMAIL, **0.94** on MAC, and over **0.90** on USERAGENT, Location, and URL. Table I should be read as a sanitization-quality proxy, not as a complete end-to-end measure of prompt-injection, poisoned-tool, or multi-step compositional attack resistance.

C. Blockchain Performance in LAN & WAN

To answer **RQ2**, we measure latency and throughput in LAN (intra-region datacenter) and WAN (geo-distributed EC2) settings, varying fault tolerance from $f = 1$ to $f = 10$.

Latency Analysis. As shown in Fig. 6, without LLM inference, LAN reads served locally remain around 40 ms

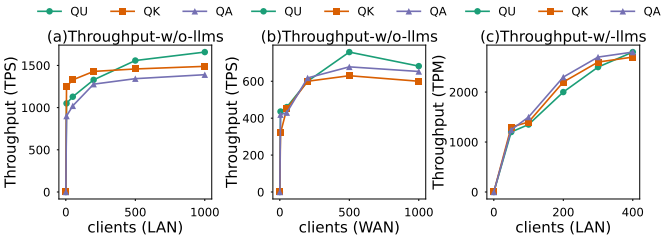

Fig. 7: Fresco Throughput.

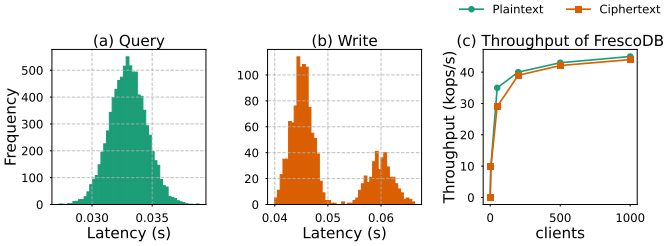

Fig. 8: Latency distribution and throughput in FrescoDB.

across all f, while writes (BFT ordering and ledger commit) increase from 50 ms at $f = 1$ to roughly 200 ms at $f = 10$. In WAN, reads average around 0.8 s and writes range from 0.9–1.2 s. With LLMs in the loop, end-to-end latency is 1.5–2.3 s depending on the model and hardware (Fig. 6(f)), indicating that inference dominates overall latency.

Throughput Evaluation. Without LLMs, Fresco reaches up to 1,500 TPS in LAN and 600 TPS in WAN (Figs. 7(a)–(b)). With LLMs enabled, we report TPM; Fresco sustains 1500–3000 TPM in LAN (Fig. 7(c)), consistent with typical per-request inference times.

D. Evaluation of On-chain Historic Auditing

Latency and Throughput of FrescoDB. To answer **RQ3**, we evaluate FrescoDB under concurrent access. Read and write latencies remain tightly distributed between 30–35 ms (Fig. 8(a–b)), and throughput peaks at around **40,000 operations per second** (40 kops/s) (Fig. 8(c)). Because the ledger stores compact hashes, timestamps, and authorization metadata rather than raw payloads, audit queries remain efficient while sensitive content stays off-chain.

E. Comparison with Existing LLM Frameworks

To answer **RQ4**, we compare Fresco against two common centralized orchestration stacks: OpenAI GPTs (Actions) and LangChain agents (via OpenAI `function_calling`). We run the same medical-record retrieval, summarization, and update workloads under benign and adversarial prompts to measure policy bypass and overhead. Here, *bypass* means that a malicious prompt still causes unauthorized tool use or protected-record disclosure; operationally, it is the workload-level false-negative view of the end-to-end defense.

Security Comparison. GPTs and LangChain provide limited tool-policy enforcement and no verifiable decision logs. Fresco instead applies scoped authorization, semantic policy checks, and on-chain audit logging. Across 500 prompts, bypass rates were ∼42% (GPTs), ∼51% (LangChain), and ∼2%

TABLE II: Latency and Policy Bypass Rate Comparison

System	Latency (ms)	Bypass Rate	On-Chain Audit?
GPTs (Actions)	∼400	∼42%	No
LangChain	∼420	∼51%	No
Fresco (ours)	∼650	∼2%	Yes

(Fresco). Because these systems embody different trust and deployment models, we use this comparison as an indicative workload-level baseline rather than a claim of architectural equivalence.

Performance Overhead. Table II shows that Fresco adds moderate latency overhead, mainly from enclave attestation, endorsement, and chain commitment. In return, it provides post-hoc reconstruction of authorization and policy decisions, which the centralized baselines do not natively support. We do not claim that this section substitutes for ablation or adaptive end-to-end attack evaluation; rather, it quantifies the current system-level tradeoff.

VIII. Discussion

Deployment feasibility. Fresco is designed as a drop-in MCP replacement: developers keep the client-side MCP interface and only replace the server endpoint with the contract-governed backend. Section VII shows up to 1,500 TPS in LAN settings with stable latency under concurrent invocations, suggesting practical deployment for enterprise settings that require auditability and policy enforcement.

Trust-model transition. Replacing a centralized MCP server with enclave-backed validators changes the trust assumption from one administrative domain to collectively verified execution. Tool selection, invocation approval, and policy enforcement become quorum-validated and immutably logged rather than opaque server-side decisions. This shift improves resilience against insider abuse and provides a clearer forensic record when policy decisions are disputed.

LLM-based policy enforcement. Fresco's semantic policy checker is designed as a controlled systems component rather than an unconstrained model call: signed model artifacts are distributed to peers, inference runs inside isolated policy containers, and endorsement records bind each decision to invocation metadata. This does not eliminate model error, but it makes policy evaluation reproducible, auditable, and harder to bypass than centralized tool orchestration without logging.

IX. Related Work

The MCP has emerged as a standardized communication bridge that enables LLMs to interact with external tools, data sources, and services in a modular and composable way. By abstracting heterogeneous backends behind a unified interface, MCP simplifies system integration and improves scalability across various enterprise workflows [7], [22], [34], [17]. Recent studies have emphasized the potential of MCP to replace ad hoc connectors with reusable microservices [9], helping LLMs move beyond isolated information silos.

However, this growing interconnectivity introduces significant security and privacy risks. Existing work has categorized MCP-related threats into several key areas. First, identity and permission mismanagement is prevalent: many MCP implementations lack cryptographic verification of tool identities and rely on insecure metadata, making them vulnerable to impersonation and overprivileged access [35], [37], [19], [36], [33], [31], [23], [12]. Second, prompt injection and tool poisoning attacks represent an increasingly critical threat vector. These attacks embed hidden malicious instructions into prompts or capability descriptions, hijacking downstream behavior and causing unauthorized tool use or data leakage [24], [35], [19], [32], [36], [28], [29], [44], [42], [13]. High-profile corporate leakage incidents have further underscored these risks [2], [1]. Third, compositional vulnerabilities arise from complex interactions between MCP clients, tools, and models. Even when each component is independently secure, their integration can produce emergent attack surfaces such as cross-connector injection [30], cascading tool invocations, or infection propagation in multi-agent systems.

To address these issues, the research community has proposed several technical countermeasures. One direction is to incorporate blockchain security primitives such as immutability, consensus, and auditability into tool-validation and context-logging workflows [4], [21], [43]. Complementary work explores how LLMs can help detect smart contract vulnerabilities, refine code, and generate tests [38], [16], [45].

Finally, multilingual safety is gaining attention as LLMs are increasingly deployed in global applications. Studies have shown that safety guarantees can degrade significantly in non-English settings, allowing attackers to bypass filters or obfuscate malicious content in other languages [40]. These challenges point to the need for more robust and language-agnostic defenses in MCP-driven workflows.

Despite this progress, most existing work addresses only part of the MCP security stack. Prior studies either characterize MCP-specific risks, improve model-side prompt-injection detection, or explore blockchain and TEE mechanisms in adjacent trusted-computing settings. Enterprise controls such as centralized gateways, certificate-based authentication, and audit logs can address parts of this problem, but they still concentrate trust in one operator and do not by themselves provide shared state integrity across mutually distrustful organizations. Fresco therefore targets a narrower systems question: how to combine authenticated tool access, decentralized policy enforcement, auditable execution logging, and privacy-preserving storage in one MCP-compatible design.

X. Conclusion

In this paper, we presented Fresco, a blockchain-based framework for partially trusted enterprise MCP deployments. Fresco combines identity verification, least-privilege access control, invocation sanitization, and auditable storage with a permissioned blockchain, BFT consensus in trusted execution environments, a PDL, and threshold-encrypted off-chain data. Our prototype and evaluation show that this design improves

accountability and policy enforcement while maintaining practical performance. Because Fresco preserves the standard MCP interaction model, these guarantees can be introduced without changing the client-side programming interface.

Fresco also has clear limitations. Its security argument depends on enclave integrity, correct policy configuration, and the quality of model-assisted sanitization; it does not yet provide full ablations or comprehensive end-to-end evaluation of adaptive compositional attacks. Larger deployments may also require adaptive committee sizing, cheaper enclave-backed operation, and richer workflow recovery.

Acknowledgement

This research was supported by the National Science Foundation under Award Number 2326034.

References

[1] Samsung bans chatgpt among employees after sensitive code leak, 2023.
[2] Samsung employees accidentally leaked company secrets via chatgpt: Here's what happened, April 2023.
[3] Create additional isolation to further protect highly sensitive data within ec2 instances, August 2025.
[4] Tarek Abdellatif, Djamel Tandjaoui, and Abdelfettah Ayed. Trustbft: Trustworthy byzantine fault tolerance for trustzone-enabled iot. In *PerCom Workshops*, 2018.
[5] M Al-Bassam, A Sonnino, S Bano, D Hrycyszyn, and G Danezis. Chainspace: A sharded smart contracts platform. In *NDSS*. Internet Society, February 2018. 25th Annual NDSS Symposium, San Diego, CA, Feb 18–21, 2018.
[6] E Androulaki, A Barger, V Bortnikov, C Cachin, K Christidis, A De Caro, D Enyeart, C Ferris, G Laventman, Y Manevich, and et al. Hyperledger fabric: A distributed operating system for permissioned blockchains. In *EuroSys*, pages 1–15, Porto, Portugal, 2018. ACM.
[7] Anthropic. Introducing the model context protocol, 2024. Anthropic News.
[8] Joppe Bos, Léo Ducas, Eike Kiltz, Tancrède Lepoint, Vadim Lyubashevsky, John M Schanck, Peter Schwabe, Gregor Seiler, and Damien Stehlé. Crystals-kyber: a cca-secure module-lattice-based kem. In *2018 IEEE European Symposium on Security and Privacy (EuroS&P)*, pages 353–367. IEEE, 2018.
[9] Alessio Bucaioni et al. A functional software reference architecture for llm-integrated systems. *arXiv preprint arXiv:2501.12904*, 2025.
[10] S. Chen, J. Piet, C. Sitawarin, and D. Wagner. Struq: Defending against prompt injection with structured queries. In *USENIX Security*, 2025.
[11] Yunang Chen, Amrita Roy Chowdhury, Ruizhe Wang, Andrei Sabelfeld, Rahul Chatterjee, and Earlence Fernandes. Data privacy in trigger-action systems. In *IEEE S&P*, pages 501–518, 2021.
[12] Cloudflare. Build and deploy remote model context protocol (mcp) servers to cloudflare, 2025.
[13] Jan Clusmann, Dyke Ferber, Isabella C Wiest, Carolin V Schneider, Titus J Brinker, Sebastian Foersch, Daniel Truhn, and Jakob Nikolas Kather. Prompt injection attacks on vision language models in oncology. *Nature Communications*, 16:1239, 2025.
[14] Jérémie Decouchant, David Kozhaya, Vincent Rahli, and Jiangshan Yu. DAMYSUS: Streamlined BFT consensus leveraging trusted components. In *EuroSys*, pages 1–16. ACM, 2022.
[15] T. Dong, M. Xue, G. Chen, R. Holland, Y. Meng, S. Li, Z. Liu, and H. Zhu. The philosopher's stone: Trojaning plugins of large language models. In *NDSS*, 2025.
[16] Qi Guo, Junming Cao, Xiaofei Xie, Shangqing Liu, Xiaohong Li, Bihuan Chen, and Xin Peng. Exploring the potential of chatgpt in automated code refinement: An empirical study. In *ICSE*, pages 34:1–34:13, 2024.
[17] Xinyi Hou, Yanjie Zhao, Shenao Wang, and Haoyu Wang. Model context protocol (mcp): Landscape, security threats, and future research directions. *arXiv preprint arXiv:2503.23278*, March 2025.
[18] H Howard, F Alder, E Ashton, A Chamayou, S Clebsch, M Costa, A Delignat-Lavaud, C Fournet, A Jeffery, M Kerner, and et al. Confidential consortium framework: Secure multiparty applications with confidentiality, integrity, and high availability. *VLDB*, 17(2), 2023.
[19] Invariant Labs. Mcp security notification: Tool poisoning attacks. Invariant Labs Blog, April 2025.
[20] D. S. Jegan, M. Swift, and E. Fernandes. Architecting trigger-action platforms for security, performance and functionality. In *NDSS*, 2024.
[21] Ramakrishna Kotla, Lorenzo Alvisi, Mike Dahlin, Allen Clement, and Edmund Wong. Zyzzyva: Speculative byzantine fault tolerance. In *SOSP*, 2007.
[22] Kseniase. What is mcp, and why is everyone – suddenly!– talking about it?, March 2025. Hugging Face.
[23] SentinelOne Labs. Avoiding mcp mania: How to secure the next frontier of ai, 2025.
[24] Ravie Lakshmanan. Researchers demonstrate how mcp prompt injection can be used for both attack and defense, April 2025.
[25] D Lee and M Tiwari. Prompt infection: Llm-to-llm prompt injection within multi-agent systems. *arXiv preprint arXiv:2410.07283*, 2024.
[26] Chenxing Li, Peilun Li, Dong Zhou, Zhe Yang, Ming Wu, Guang Yang, Wei Xu, Fan Long, and Andrew Chi-Chih Yao. A decentralized blockchain with high throughput and fast confirmation. In *USENIX ATC*, pages 515–528. USENIX Association, July 2020.
[27] Jian Liu, Wenting Li, Ghassan O. Karame, and N. Asokan. Scalable byzantine consensus via hardware-assisted secret sharing. *IEEE Trans. Comput*, 2018. FastBFT uses TEEs to aggregate messages and reach 100k+ TPS. doi:10.1109/TC.2018.2860009.
[28] Yi Liu, Gelei Deng, Yuekang Li, Kailong Wang, Tianwei Zhang, Yepang Liu, Haoyu Wang, Yan Zheng, and Yang Liu. Prompt injection attack against llm-integrated applications, 2023.
[29] Yupei Liu, Yuqi Jia, Runpeng Geng, Jinyuan Jia, and Neil Zhenqiang Gong. Formalizing and benchmarking prompt injection attacks and defenses. In *USENIX Security*, 2024.
[30] Microsoft Defender for Cloud. Cross-connector attacks: New threat vectors in ai-driven workflows, 2024. Accessed: 2025-05-12.
[31] Vineeth Sai Narajala and Idan Habler. Enterprise-grade security for the model context protocol (mcp): Frameworks and mitigation strategies. *arXiv preprint arXiv:2504.08623*, April 2025.
[32] Phala Network. Mcp not safe—reasons and ideas, April 2025.
[33] Brandon Radosevich and John Halloran. Mcp safety audit: Llms with the model context protocol allow major security exploits. *arXiv preprint arXiv:2504.03767*, April 2025.
[34] M. Rajguru. Model context protocol (mcp): Integrating azure openai for enhanced tool integration and prompting, March 2025. Microsoft Azure AI Services Blog.
[35] Dor Sarig. The security risks of model context protocol (mcp), 2025.
[36] Kasimir Schulz, Jason Martin, Marcus Kan, Kenneth Yeung, Conor McCauley, and Leo Ring. Mcp: Model context pitfalls in an agentic world, April 2025.
[37] Upwind Security. Unpacking the security risks of mcp servers, 2024.
[38] Yuqiang Sun, Daoyuan Wu, Yue Xue, Han Liu, Haijun Wang, Zhengzi Xu, Xiaofei Xie, and Yang Liu. GPTScan: Detecting logic vulnerabilities in smart contracts by combining GPT with program analysis. In *ICSE*, pages 1–11 (to appear), 2024.
[39] Le Wang, Zonghao Ying, Tianyuan Zhang, Siyuan Liang, Shengshan Hu, Mingchuan Zhang, Aishan Liu, and Xianglong Liu. Manipulating multimodal agents via cross-modal prompt injection. *arXiv preprint arXiv:2504.14348*, 2025.
[40] Wenxuan Wang, Zhaopeng Tu, Chang Chen, Youliang Yuan, Jen-tse Huang, Wenxiang Jiao, and Michael Lyu. All languages matter: On the multilingual safety of LLMs. In *ACL Findings*, pages 5865–5877. Association for Computational Linguistics, 2024.
[41] Lei Yang, Vivek Bagaria, Gerui Wang, Mohammad Alizadeh, David Tse, Giulia Fanti, and Pramod Viswanath. Prism: Scaling bitcoin by 10,000×. In *Stanford Blockchain Conference*, 2020.
[42] Jingwei Yi, Yueqi Xie, Bin Zhu, Emre Kiciman, Guangzhong Sun, Xing Xie, and Fangzhao Wu. Benchmarking and defending against indirect prompt injection attacks on large language models. In *KDD*, 2025.
[43] Maofan Yin, Dahlia Malkhi, Michael K Reiter, Guy Gueta, and Ittai Abraham. Hotstuff: Bft consensus with linearity and responsiveness. In *PODC*, 2019.
[44] Sarah Young and Den Delimarsky. Protecting against indirect prompt injection attacks in mcp. Microsoft DevBlogs, April 2025.
[45] Zhiqiang Yuan, Mingwei Liu, Shiji Ding, Kaixin Wang, Yixuan Chen, Xin Peng, and Yiling Lou. Evaluating and improving chatgpt for unit test generation. In *FSE*, volume 1, pages Article 76, 24 pages, 2024.